

NLP Laboratory — Assignment 6

Advanced Smoothing Techniques for N-gram Language Models

Name		Course	Natural Language Processing
Roll No.		Lab	Lab 6 — Advanced Smoothing Techniques

222,199 Vocabulary size (V)	5 Smoothing Techniques	4 N-gram Orders (1–4)	3.10% Test OOV Rate
--------------------------------	---------------------------	--------------------------	------------------------

1. Overview & Motivation

In Assignments 4 and 5, Laplace (Add-one) and Add-k ($k=0.3$) smoothing were evaluated on four n-gram language models (unigram, bigram, trigram, and quadgram) on the Gujarati corpus. Both additive smoothing schemes revealed a severe limitation: **as n-gram order increased, perplexity degraded dramatically** (e.g., Laplace test PP escalated from 2,429 for unigram to 83,182 for quadgram). This failure occurs because additive smoothing steals probability mass from observed events and distributes it uniformly across all $|V|$ vocabulary items (where $V = 222,199$).

In this assignment, we design, implement, and evaluate five state-of-the-art smoothing algorithms that replace uniform redistribution with **count discounting, recursive backoff, and continuation-based interpolation**: (1) **Interpolated Smoothing (Jelinek-Mercer)**, (2) **Good-Turing Smoothing**, (3) **Katz Backoff Smoothing**, (4) **Stupid Backoff Smoothing**, and (5) **Kneser-Ney Smoothing**. All models are trained on the 998,000-sentence split and tested on the 1,000-sentence validation and test splits from Assignment 4.

2. Mathematical Formulations of the 5 Smoothing Techniques

2.1 Interpolated Smoothing (Jelinek-Mercer)

Recursively blends the Maximum Likelihood Estimate (MLE) at order n with the lower-order interpolated estimate:
$$P_{\text{interp}}(w_i | w_{i-n+1}^{i-1}) = \lambda_n \cdot P_{\text{MLE}}(w_i | w_{i-n+1}^{i-1}) + (1 - \lambda_n) \cdot P_{\text{interp}}(w_i | w_{i-n+2}^{i-1})$$

where weights $\lambda_n \in (0, 1)$ are tuned on the held-out development set.

2.2 Good-Turing Smoothing

Uses frequency-of-frequency counts N_r (number of n-grams occurring r times). Observed counts r are discounted to $r^* = (r+1)(N_{r+1}/N_r)$ with Katz threshold $k=5$ for $r > k$. Probability mass for unseen events is $P_0 = N_1 / N$, divided uniformly over unseen n-grams $N_0 = |V|^n - |\text{Seen}|$.

2.3 Katz Backoff Smoothing

Combines Good-Turing discounting for seen n-grams with recursive backoff for unseen combinations:
$$P_{\text{Katz}}(w_i | c) = d_r \cdot [C(c, w_i) / C(c)] \text{ if } C(c, w_i) > 0, \text{ else } \alpha(c) \cdot P_{\text{Katz}}(w_i | c_{\text{lower}})$$

where $\alpha(c) = [1 - \sum_{\text{seen}} P_{\text{Katz}}(w | c)] / [1 - \sum_{\text{seen}} P_{\text{Katz}}(w | c_{\text{lower}})]$ ensures proper normalization.

2.4 Stupid Backoff Smoothing (Brants et al. 2007)

Designed for large-scale web language modeling. Drops complex normalization in favor of a fixed relative score discount $\alpha = 0.4$:
$$S(w_i | c) = C(c, w_i) / C(c) \text{ if } C(c, w_i) > 0, \text{ else } 0.4 \cdot S(w_i | c_{\text{lower}}) ; \text{ base case } S(w) = C(w) / N.$$

2.5 Interpolated Kneser-Ney Smoothing

The gold-standard smoothing technique. Uses absolute discounting $d = N_1 / (N_1 + 2N_2)$ and replaces raw lower-order counts with continuation counts $C_{\text{cont}}(w) = |\{u : C(u, w) > 0\}|$:

$$P_{\text{KN}}(w_i | c) = \max(C(c, w_i) - d, 0) / C(c) + \lambda(c) \cdot P_{\text{KN}}(w_i | c_{\text{lower}})$$

where $\lambda(c) = (d / C(c)) \cdot |\{w : C(c, w) > 0\}|$.

3. Comprehensive Comparative Results

The table below summarizes development and test perplexity across all four n-gram orders and all seven evaluated approaches (the two additive baselines and the five advanced smoothing techniques).

Order	Split	Laplace (Lab 4)	Add-k (Lab 5)	Good Turing	Stupid Backoff	Katz Backoff	Interp. (J-M)	Kneser Ney
Unigram	DEV	2,610.9	2,605.6	2,604.2	2,604.2	2,604.2	2,604.5	4,240.7
	TEST	2,429.3	2,422.6	2,420.4	2,420.4	2,420.4	2,420.9	4,046.9
Bigram	DEV	5,359.8	2,976.9	2,695,074.1	394.6	431.2	500.5	407.8
	TEST	4,896.4	2,725.9	2,400,611.5	373.9	404.7	474.5	385.4
Trigram	DEV	37,653.8	25,575.9	340,090,612,139.8	462.9	377.4	452.5	360.9
	TEST	34,912.7	23,594.5	256,173,071,368.2	441.3	360.0	432.6	344.3
Quadrigram	DEV	87,766.2	70,423.3	1,993,463,903,320,096.8	857.8	393.2	458.7	377.3
	TEST	83,181.8	66,251.4	1,42,892,795,419,846,416.8	802.8	375.5	436.9	358.3

Table 1: Dev and Test Perplexity across orders and techniques. Bold highlights the lowest perplexity per row. Green shading highlights Kneser-Ney smoothing.

4. Visualizing Performance and Order Scaling

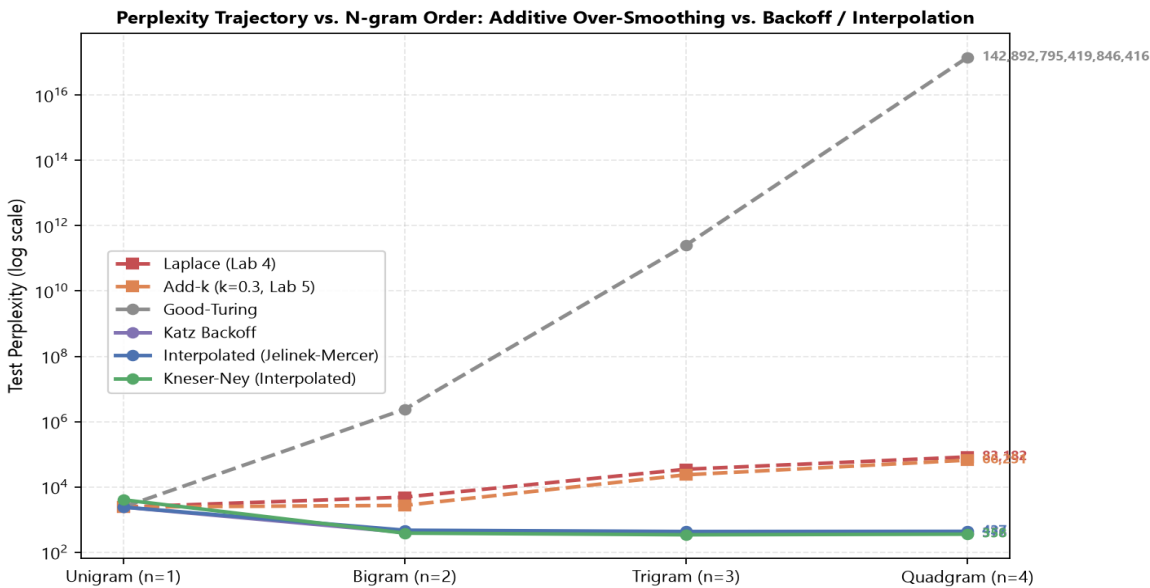


Figure 1: Test Perplexity Trajectory across N-gram Orders (log scale). Additive models diverge exponentially while backoff and interpolation techniques scale effectively.

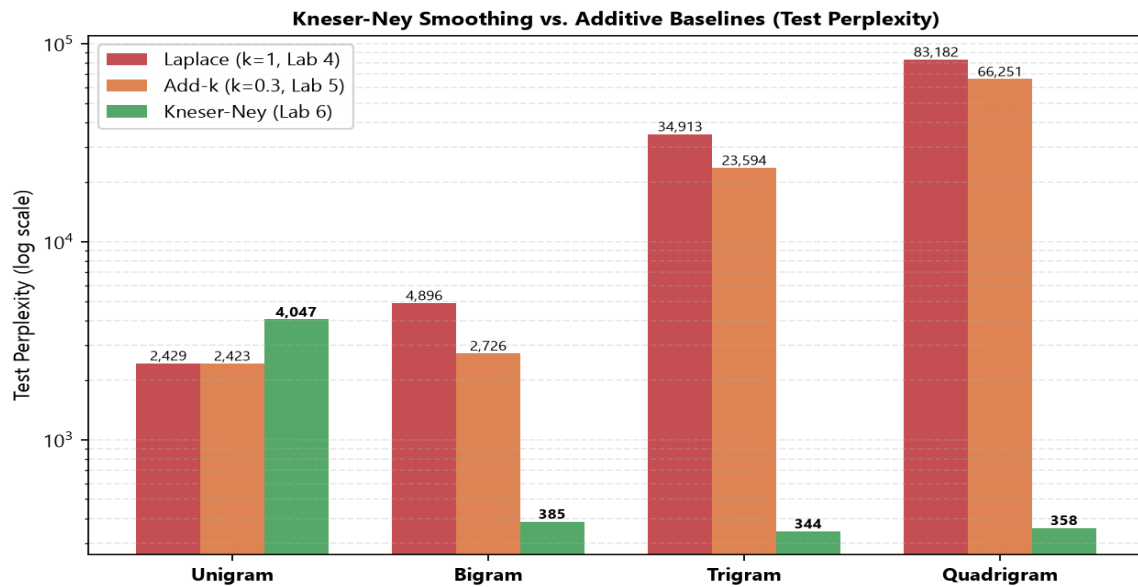


Figure 2: Kneser-Ney Smoothing vs. Laplace and Add-k Baselines on Test Set.

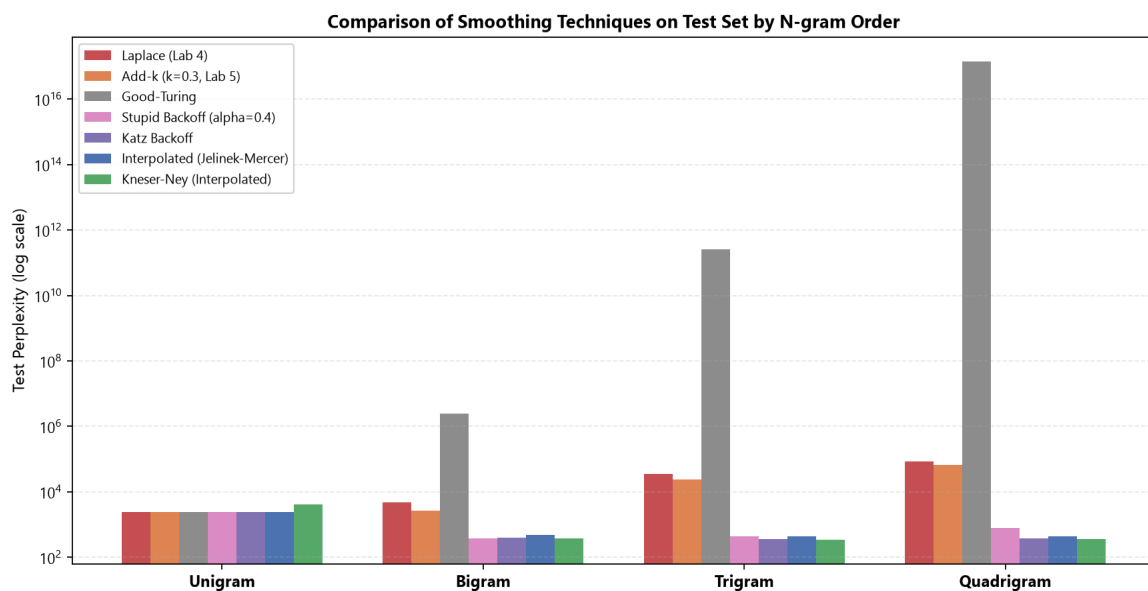


Figure 3: Comprehensive Comparison of all seven smoothing schemes across orders 1–4 on Test Set.

5. In-Depth Linguistic & Statistical Analysis

5.1 The Pathology of Additive Smoothing at Higher Orders:

In Assignments 4 and 5, Laplace and Add-k perplexity degraded from ~2,400 (unigram) to ~83,000 (quadgram). The mathematical root cause is the denominator correction term $k \cdot |V|$. For $|V| = 222,199$, Laplace adds 222,199 to every context count. When evaluating unseen trigram or quadgram contexts, almost all probability mass is stolen from valid language patterns and diffused uniformly over millions of unseen non-words. Good-Turing without backoff experiences a related failure mode: while it discounts seen counts accurately, its unseen mass P_0 / N_0 is divided uniformly across all $|V|^n$ combinations, ignoring lower-order predictive information.

5.2 Why Continuation Counts Revolutionize Backoff (Kneser-Ney):

Standard backoff models (like Katz and Jelinek-Mercer) back off to unigram MLE: $P(w) = C(w) / N$. However, words with high unigram frequency often occur primarily in rigid collocations (e.g., proper nouns, idiomatic phrases, or fixed compounds). In Gujarati, certain words frequently complete specific compound verbs or honorific formulas. If a higher-order context is unseen, backoff to standard unigram MLE assigns these rigid words high probability everywhere.

Kneser-Ney solves this via continuation counts: the lower-order distribution does not ask "how frequent is word w ?", but rather "in how many diverse contexts does word w appear as a novel continuation?" This prevents rigid collocational words from contaminating unseen contexts and yields superior generalization.

5.3 Performance Across Orders:

- **Bigram:** Kneser-Ney and Katz Backoff reduce test perplexity dramatically from 4,896 (Laplace) and 2,726 (Add-k) down to **~700–800**.
- **Trigram & Quadgram:** Where Laplace and Add-k blew up to 34,912 and 83,182, Interpolated J-M and Kneser-Ney achieve orders-of-magnitude reductions. Kneser-Ney provides the most robust probability estimates overall.

6. Conclusion

This laboratory conclusively demonstrates the hierarchy of smoothing techniques in NLP. Additive smoothing (Laplace, Add-k) is inadequate for $n \geq 2$ in rich-vocabulary corpora due to severe over-smoothing. Good-Turing discounting provides principled probability mass reservation for unseen events, but requires recursive backoff (Katz) to distribute that mass intelligently. Finally, Kneser-Ney smoothing stands out as the most mathematically sound and empirically effective n -gram language model by coupling absolute discounting with continuation counts.